aie26

Topic 08 : Classification1

Part 02 : Logistic Regression

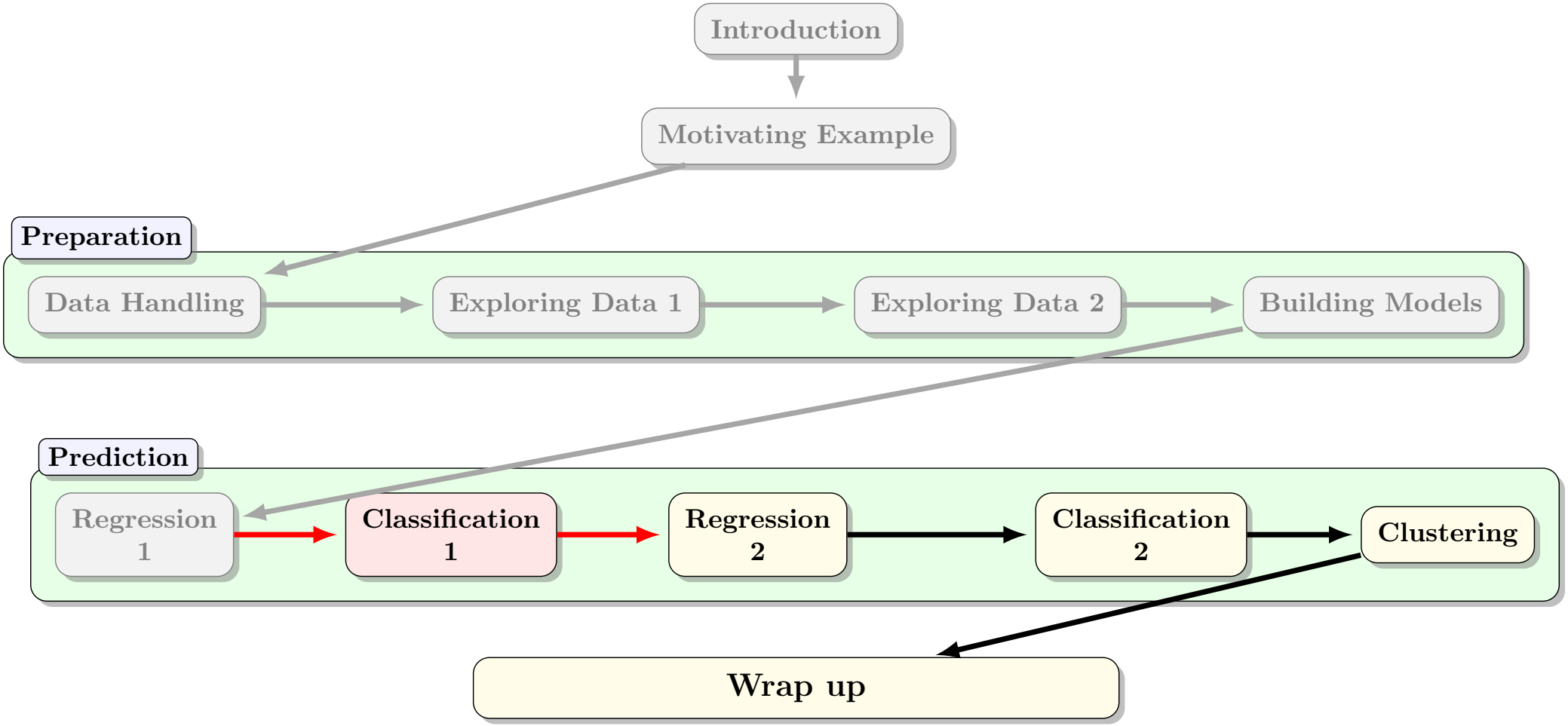
Dr Bernard Butler
Department of Computing and Mathematics, SETU.
(bernard.butler@setu.ie)

Autumn Semester, 2026

Outline

- Logistic regression in practice - Iris dataset
- Logistic regression - how it works
- ROC curves and their usage
- Feature engineering, including correlation analysis

AI Engineering (Week 8)



Outline

1. IRIS Dataset — Classification using Logistic Regression 4

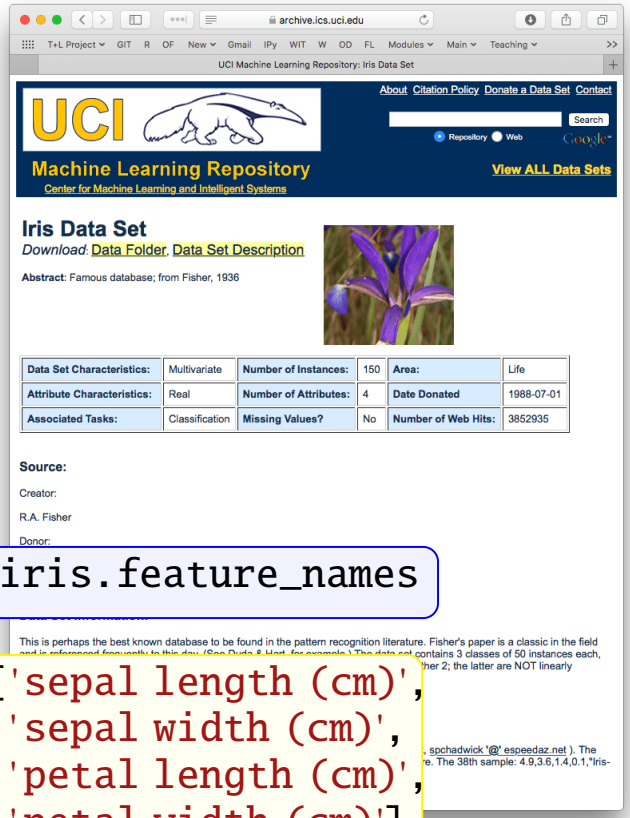
2. Logistic Regression 11

3. Area under the Curve 18

4. Features and Dimensions 25

5. Review 33

Example: IRIS Dataset — Load




```
iris.feature_names
```

```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

```
from sklearn import datasets
iris = datasets.load_iris()

df = pd.DataFrame(iris.data)
df.columns = iris.feature_names
df['target'] = iris.target_names[iris.target]
df.sample(4)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
25	5.0	3.0	1.6	0.2	setosa
21	5.1	3.7	1.5	0.4	setosa
113	5.7	2.5	5.0	2.0	virginica
27	5.2	3.5	1.5	0.2	setosa

The data set contains, four numeric features, 3 classes of 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

Example: IRIS Dataset — Preprocess Data

We will cover some classifiers in a moment, but for now just treat the classifiers (`LogisticRegression`) as a black box and focus on the general process:

Extract the data (features and target)

The IRIS dataset has 4 features, but to simplify visualisation we are only going to use the first two* ('sepal length' and 'sepal width'):

```
dataset_name = "IRIS"
X, y, target_names = iris.data[:,:2], iris.target, iris.target_names
```

Split dataset into `train` and `test`

We will keep 40% of the data for testing. Setting the parameter `random_state` to a value means that we will get a random — but still reproducible — split.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.6, random_state=666)
```

:

*Python for Data Science — Cheat Sheet Numpy Basics

Example: IRIS Dataset — Fit Model and Predict

Select classifier

Scikit-learn supports a large set of classifiers, and aims to have a consistent interface to all. First import classifier and create instance ...

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=500)
```

Train model

Then we train (fit) the classifier/model using only the features (`X_train`) and targets (`y_train`) from the train dataset ...

```
model.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=500)
```

Predict

Now that model is trained, we can use it to generate predictions, using the features (`X_test`) from the test dataset ...

```
y_pred = model.predict(X_test)
```

Example: IRIS Dataset — Evaluate I

Scoring and confusion matrix

We could just compute the score using whatever metric we have picked . . .

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

```
0.8333333333333334
```

But this needs context, and even if good can hide critical flaws. Lets look at the confusion matrix . . .

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test,y_pred)
cm
```

```
array([[18, 1, 0],
       [ 0, 17,  6],
       [ 0,  3, 15]])
```

or, to get a nicer output, convert to a DataFrame . . .

```
df_cm = pd.crosstab(target_names[y_test],target_names[y_pred])
df_cm.index.name = 'Actual'
df_cm.columns.name = 'Predicted'
df_cm
```

Predicted	setosa	versicolor	virginica
Actual			
setosa	18	1	0
versicolor	0	17	6
virginica	0	3	15

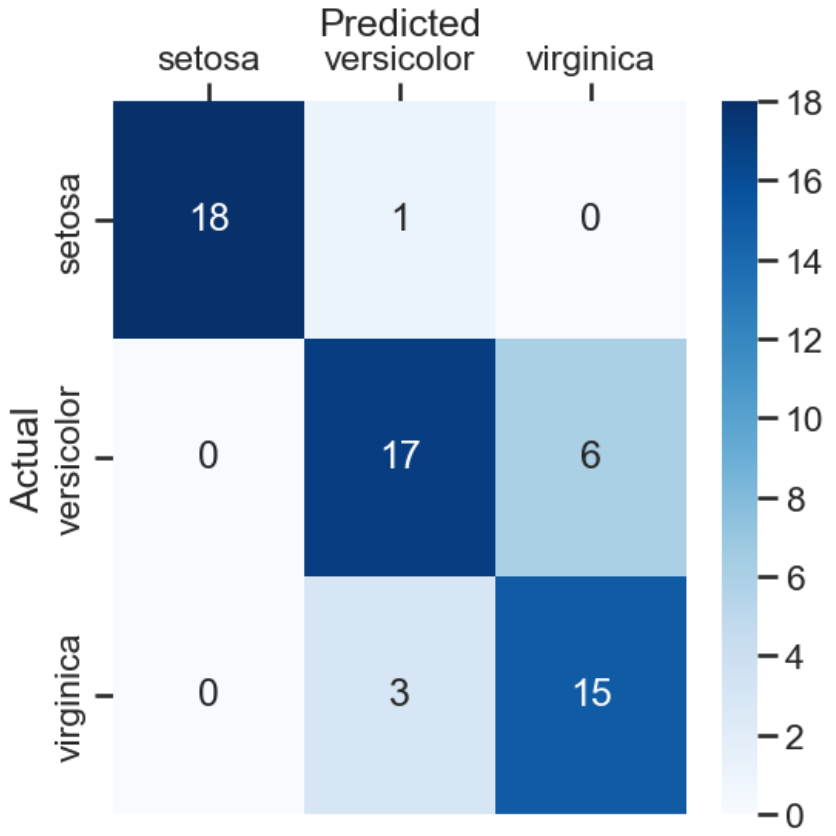
Example: IRIS Dataset — Evaluate II

The confusion matrix is fundamental in evaluating a classifier, so find a presentation/visualisation that you like and use it. Here I have a heat map representation that I tend to use.

Predicted	setosa	versicolor	virginica
Actual			
setosa	18	1	0
versicolor	0	17	6
virginica	0	3	15

The first class `setosa` was only misclassified once, while the classifier had more difficulty between the second two classes.

```
plt.figure(figsize=(6,6))
g = sns.heatmap(df_cm, annot=True, cmap="Blues")
g.xaxis.set_ticks_position("top")
g.xaxis.set_label_position('top')
```



Example: IRIS Dataset — Evaluate III

The classification report, constructed from the confusion matrix, summarises the most common metrics per class and for overall averages ...

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred, target_names=target_names))
```

precision (setosa) = $18/(18+0) = 1$

recall (setosa) = $18/(18+1) = 0.95$

Predicted / Actual	setosa	versicolor	virginica
setosa	18	1	0
versicolor	0	17	6
virginica	0	3	15

```
                precision    recall  f1-score   support

     setosa         1.00      0.95      0.97        19
 versicolor         0.81      0.74      0.77        23
  virginica         0.71      0.83      0.77        18

   accuracy                             0.83        60
  macro avg       0.84      0.84      0.84        60
weighted avg      0.84      0.83      0.84        60
```

accuracy = $(18+17+15)/60 = 0.83$

f1-score (virginica) = $2/(1/0.71 + 1/0.83) = 0.77$

Motivation

Linear regression is a very powerful and flexible prediction model (Topics 7 and 8)

Can it be used to predict categorical variables, perhaps coded as numbers?

- Pros
  - Linear regression provides a principled way of combining the contributions of the predictors, whether they are numeric or not.
  - There is a lot of well-established theory and practice, e.g., in respect of collinearity.
  - The model is extremely flexible, e.g., predictors can be nonlinear functions of the features.
  - Implementations can use computing resources efficiently.
- Cons
  - Prediction is numeric value, needs to be converted to a categorical value.
  - Conversion function introduces unwelcome features, e.g., ordering and scaling, that do not apply to nominal variables.
  - Interpretation of linear regression in terms of classification performance is tricky because the conversion function needs to balance continuity against evaluating to either 0 or 1.

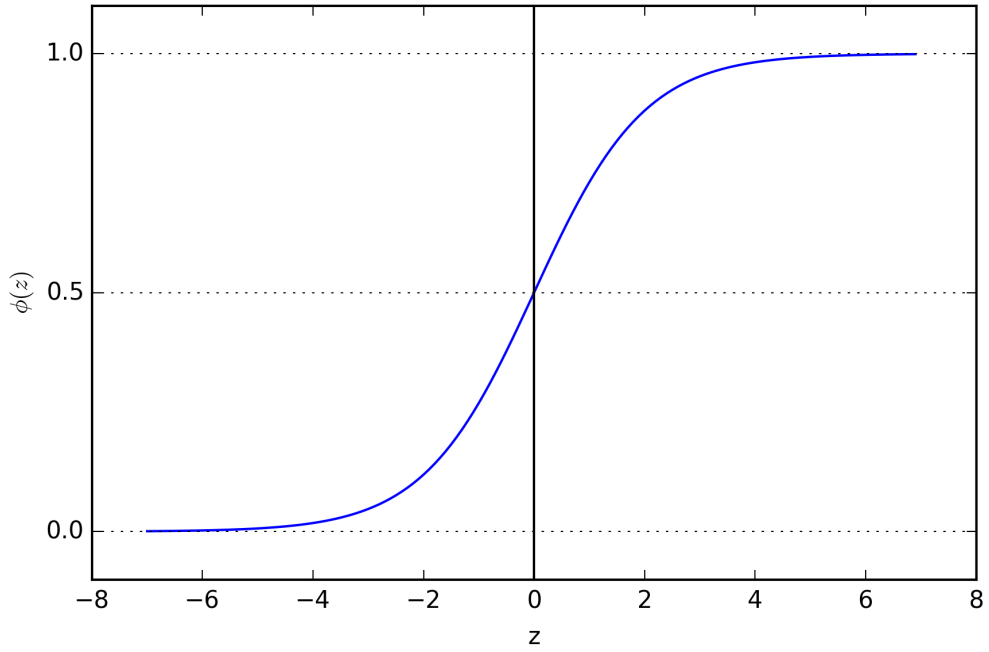
So—is there any hope for using linear regression in classification?

Probability view of Classification

When we predict a nominal value, it is equivalent to saying that the probability that a given observation takes that value is high and that it takes any other value is low.

- Probabilities are numeric, but their range is restricted to $[0, 1]$.
- We need a function of the predictors with the following properties
  - it has the range $[0, 1]$
  - it is defined over a domain which is $(-\infty, \infty)$
  - it can evaluate to 0 or 1, depending on its input.
- A line (like $y = \beta_0 + \beta_1 x$), such as we used in previous topics, does not have these properties
- Ideally, the function should be smooth and well-behaved everywhere, *and* evaluate to 0 or 1 as appropriate.
- Note that the classes are labeled as 0 or 1 (binary classification).

Introducing the logistic function



Definition 1 (logistic function)

The curve above is given by the logistic function, which can be written as $p(z) = \frac{e^z}{1+e^z}$. Letting $z = \beta_0 + \beta_1 X$, we have $p(X) = \frac{e^{\beta_0+\beta_1 X}}{1+e^{\beta_0+\beta_1 X}}$ for some β_0 and β_1.

Note that a threshold, $\tilde{\phi(z)}$, is needed, so if $\phi(z) < \tilde{\phi(z)}$, the classifier predicts the negative class, otherwise it predicts the positive class. Conventionally, $\tilde{\phi(z)} = 0.5$, but that is not always optimal, especially if there is a greater cost associated with either *false negatives* or *false positives*.

Predicting the (log) odds ratio

$$
\begin{aligned}
p(X) &= \frac{e^{\beta_0+\beta_1 X}}{1+e^{\beta_0+\beta_1 X}} && (1)\\
\implies e^{\beta_0+\beta_1 X} &= p(X) + e^{\beta_0+\beta_1 X} p(X)\\
\implies (1-p(X))e^{\beta_0+\beta_1 X} &= p(X)\\
\implies \frac{p(X)}{1-p(X)} &= e^{\beta_0+\beta_1 X}\\
\implies \log\left(\frac{p(X)}{1-p(X)}\right) &= \beta_0+\beta_1 X && (2)
\end{aligned}
$$

The expression on the left of (2) is called the *log-Odds Ratio* (or logit(X)). When $p(X) \to 1$, logit$(X) \to \infty$ and when $p(X) \to 0$, logit$(X) \to -\infty$, as required.
The expression on the right of (2) is a linear form in β, as used in linear regression.
For training: Compute logit(X) from the training labels and use linear regression to get $\{\beta_i\}$.
For prediction: Substitute X, $\{\beta_i\}$ in the logistic function (1) to obtain class **round**$(p(X))$.

Logistic regression summary

Training

Given (labeled) training data, convert the label to the equivalent logit value and use *maximum likelihood estimation* to look for the parameters β that make the observed data as likely as possible. Extension to multiple predictors is trivial.

Prediction

Given the fitted β, just evaluate the logistic function for a specific X. The resulting $p(X)$ will hopefully be near 0 or 1 and can be interpreted according to how "success" ($p = 1$) is defined.

Extension to categorical features

As with linear regression: create dummy (binary) indicator (0,1)-valued variables, one for each level of the categorical predictor.

Extension to non-binary targets

We can convert to extra indicator variables, but at some cost in complexity. Therefore, logistic regression is best suited to binary prediction.

Logistic regression in python

Python's `scikit-learn` and `statsmodel` libraries provide a general interface to model fitting that abstracts away logistic functions and other details.

Method (Recognising the Handwritten Digits)

```
from sklearn.linear_model import LogisticRegression

# Get and configure a LogisticRegression object
clf = LogisticRegression(max_iter=7600)

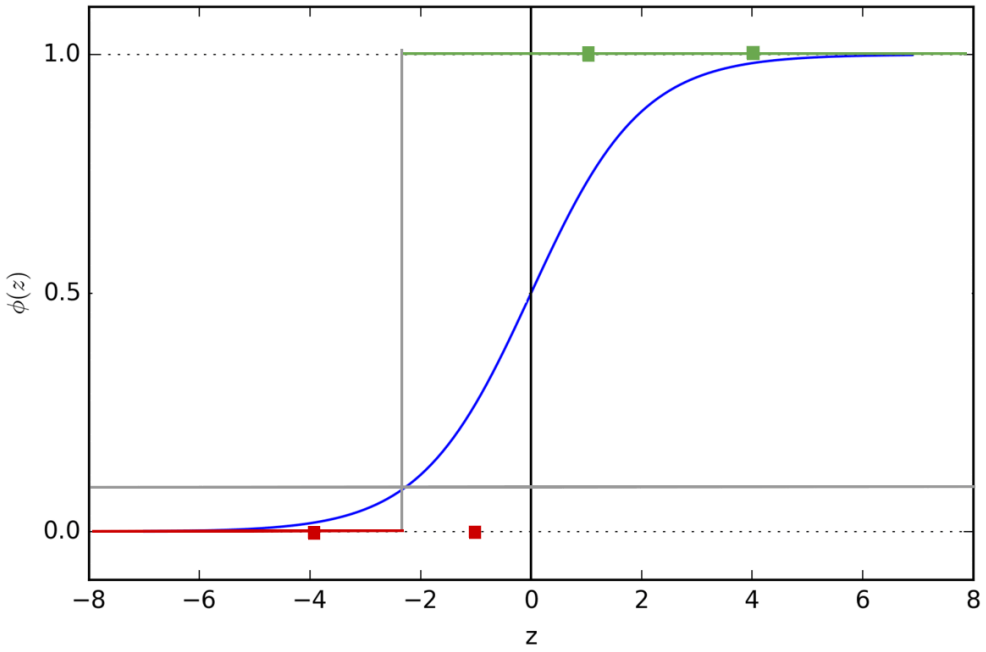
# Fit the training data
clf.fit(Xtrain, ytrain)

# Using the beta parameters that have been learned and are in clf, predict (recognise) the test data
ypred = clf.predict(Xtest)
```

Logistic regression with a low threshold

Logistic Regression requires a probability threshold

- In the plot, $\phi(z)$ is the probability that the predicted label is positive.
- If the threshold is at $\phi(z) = 0.1$, all the actual positive values are predicted correctly, but one of the negative values is misclassified (red point to the right of the z threshold).
- We have TP=2, FP=1, TN=1 and FN=0.
- Hence the True Positive Rate (TPR) = TP / (TP + FN) = 2/2 = 1.
- And the False Positive Rate (FPR) = FP / (FP + TN) = 1/2 = 0.5.

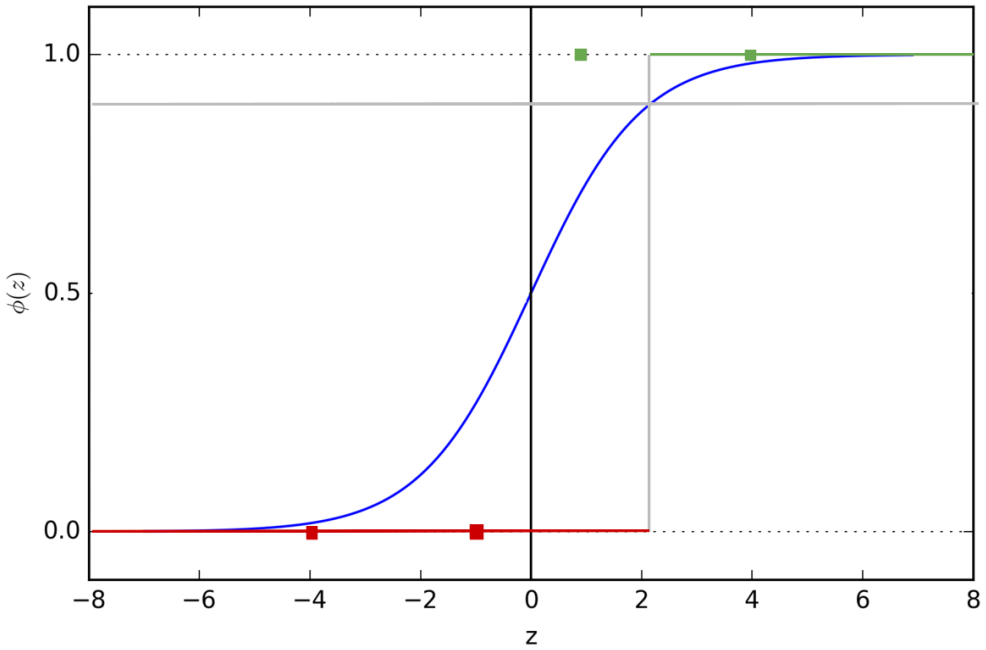


With a low threshold, the TPR is 1, but this is at the expense of increasing the FPR

Logistic regression with a high threshold

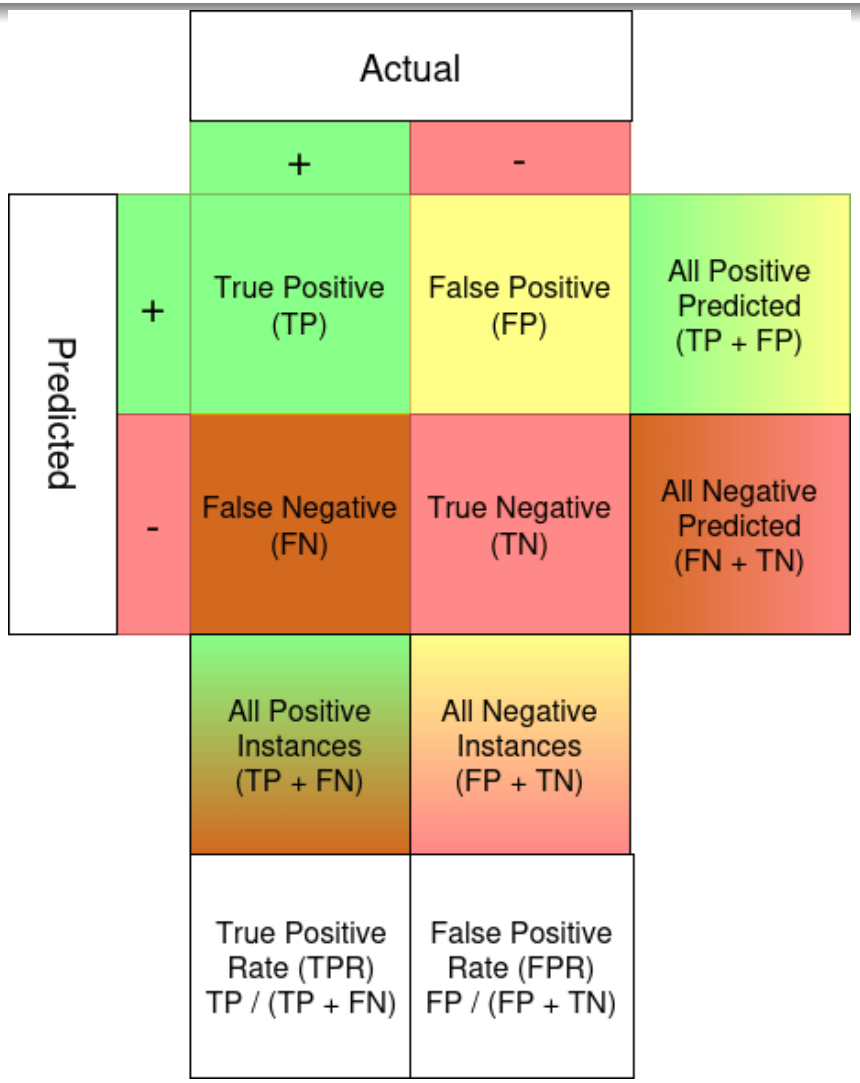
What if we increase the Logistic Regression probability threshold?

- In the plot, $\phi(z)$ is the probability that the predicted label is positive.
- If the threshold is at $\phi(z) = 0.9$, all the actual negative values are predicted correctly, but one of the positive values is misclassified (green point to the left of the z threshold).
- We have TP=1, FP=0, TN=2 and FN=1.
- Hence the True Positive Rate (TPR) = TP / (TP + FN) = 1/2 = 0.5.
- And the False Positive Rate (FPR) = FP / (FP + TN) = 0/2 = 0.



With a high threshold, the FPR is 0, but this is at the expense of decreasing the TPR

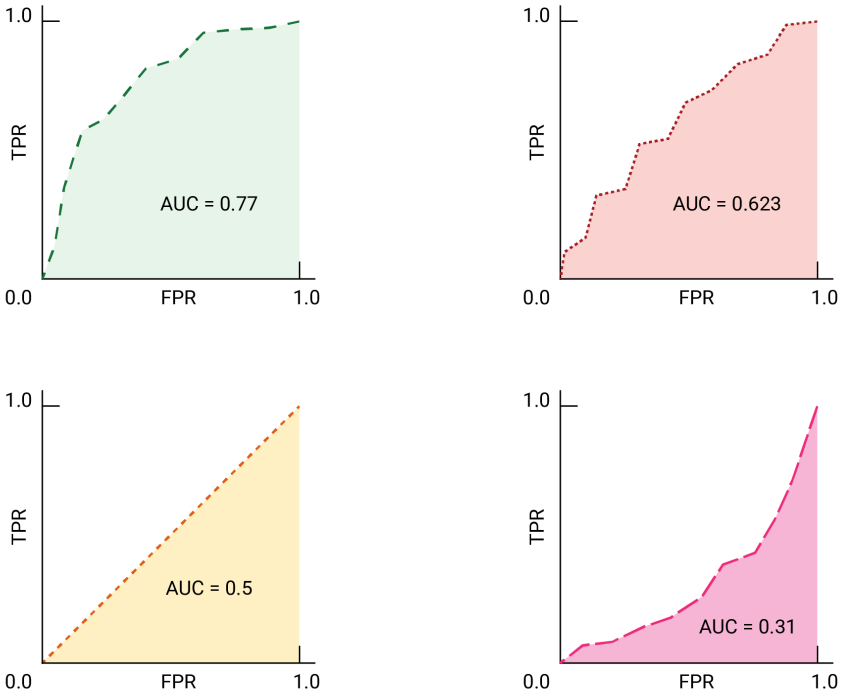
Computing TPR and FPR



- TPR measures how well the classifier, with a given threshold, assigns the positive class label to instances where that label is correct.
- High TPR is better than low TPR.
- Conversely, low FPR is better than high FPR.
- As seen, low probability thresholds increase TPR (good) but can also increase FPR (bad).
- Conversely, high probability thresholds have the opposite effect.
- How can we compare two classifiers across multiple probability thresholds?

Comparing thresholds with ROC curves and AUC

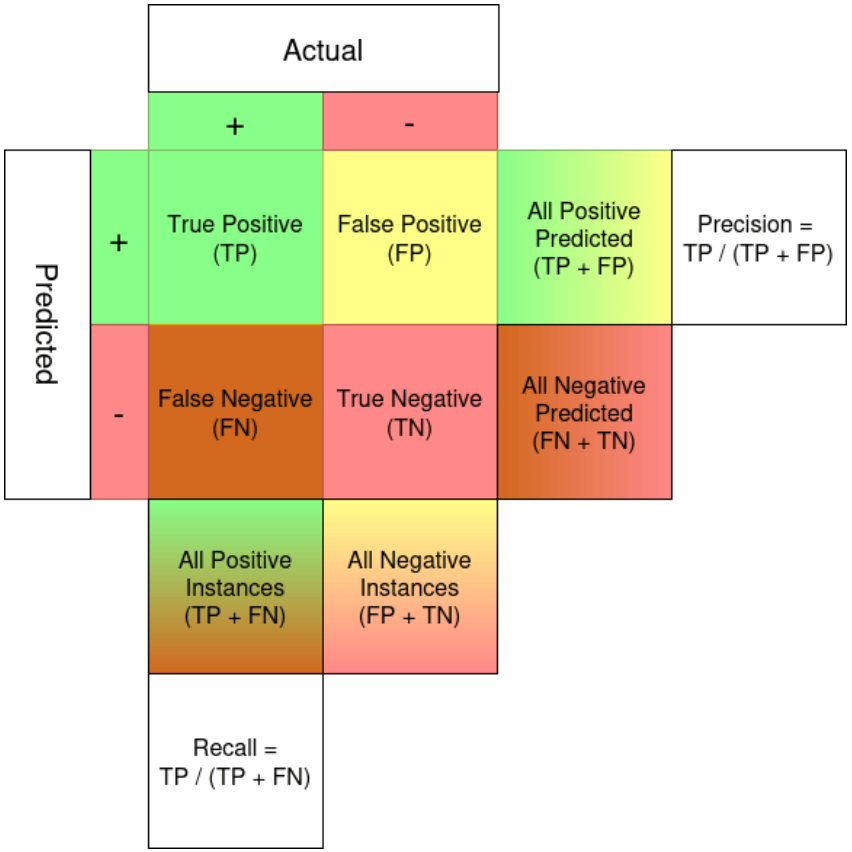
- Receiver Operating Characteristic (ROC) curves plot TPR vs FPR for probability thresholds in [0,1].
- Area Under the ROC curve (AUC) is a *threshold-independent* measure of classifier performance.
- Usually, there is a trade-off between TPR and FPR, so AUC < 1.
- The classifier with AUC $= 0.77$ performs better than the one with AUC $= 0.623$.
- Random guessing has AUC $= 0.5$.
- The classifier with AUC $= 0.31$ performs worse than chance and so is useless.



Plots from this page

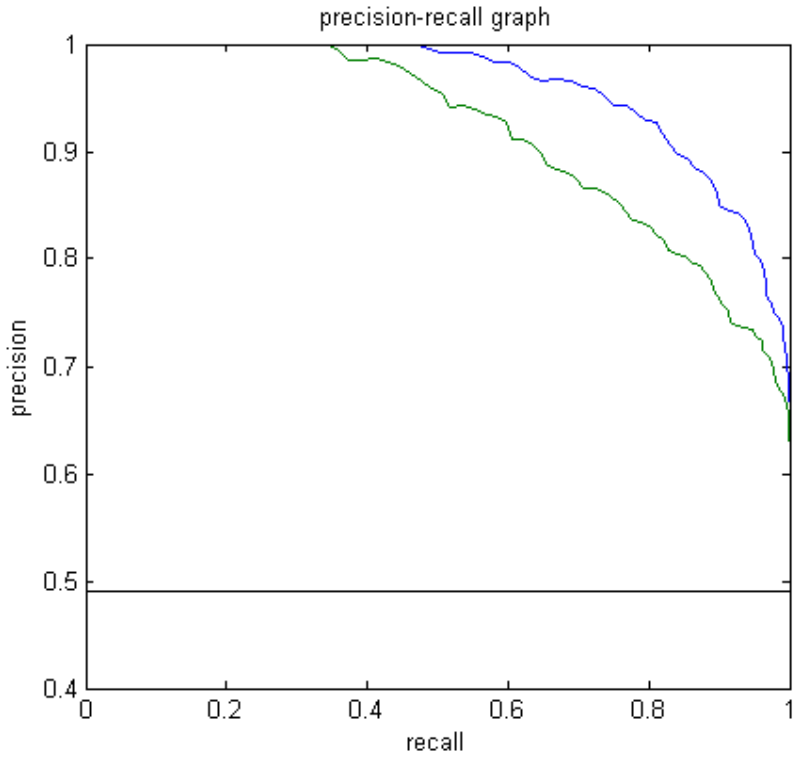
Given two classifiers, the classifier with the higher AUC generally performs better.

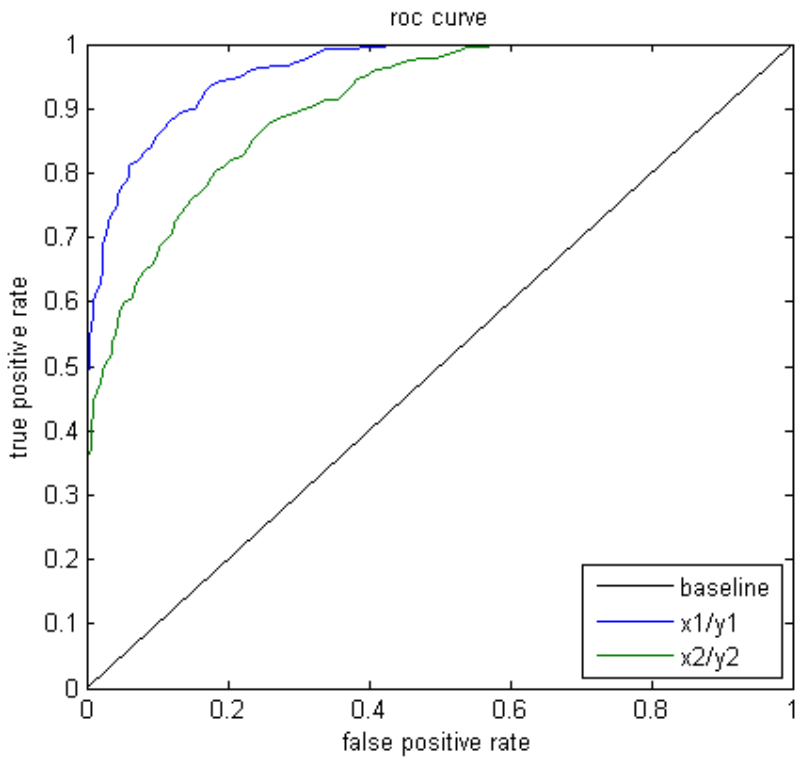
Unbalanced data - when positive instances are rare



- When positive instances are rare (e.g., fraud, ebola, . . .) but missing them is "expensive", FPR is less important than identifying the positive cases consistently (Precision).
- So we focus on Precision vs Recall instead.
- Again, there is a tradeoff between Precision and Recall when choosing the probability threshold, so we plot Precision-Recall curves instead of ROC curves.
- We can compare two classifiers using Precision-Recall and its associated AUC.

Using ROC and P-R curves



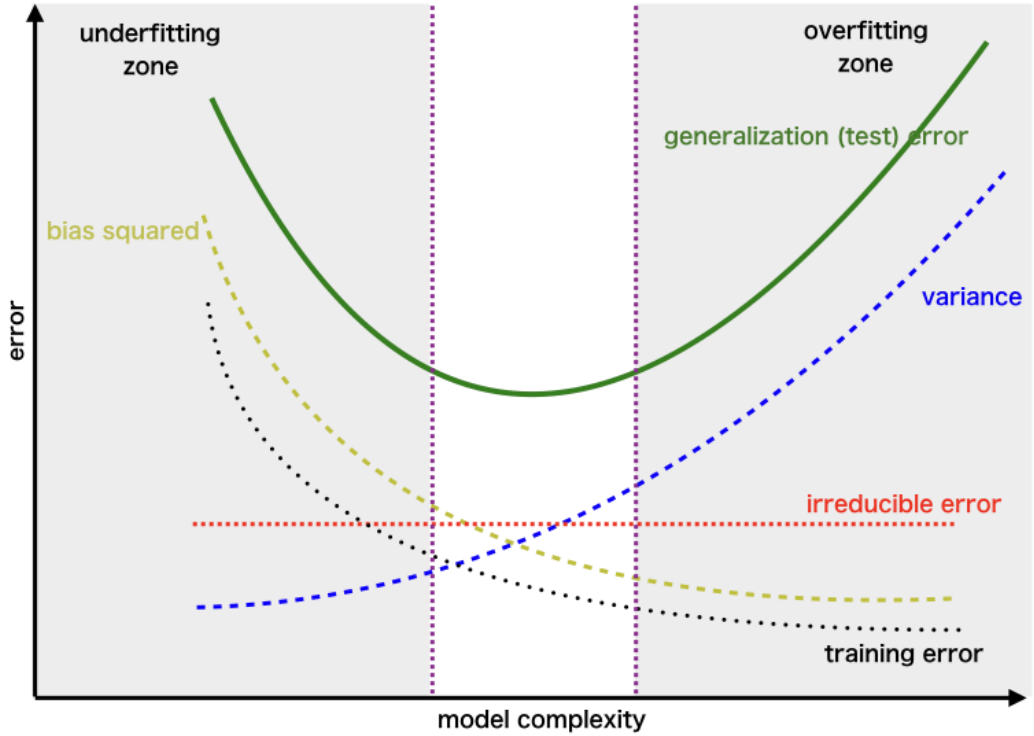


- For P-R curves, closer to the *top right* corner gives higher AUC. So "blue" classifier is better.
- For ROC curves, closer to the *top left* corner gives higher AUC. So "blue" classifier is better.

Review of model building in supervised learning

Goal: Choose the features and model formulation that predict the target, minimising error on the test set

- What features should we choose?
- Need enough to minimise bias due to underfitting
- . . . But not enough to increase variance due to overfitting
- So choose best subset (this is a combinatorial task (2^n))
- . . . But are the features in the dataset the best ones to choose from?
- Can we derive new ones that do better?



Features can be selected from an existing set, transformed from individual features, or derived from a set of features.

Explanatory power and number of features

- The relationship between features and targets can be very complex.
- If a feature $X[:, j]$ is very "simple", how valuable is it when it comes to explaining the target?
- If the target y is also "simple", *and* they are highly correlated, then feature $X[:, j]$ is very valuable
- Otherwise, it is less valuable, except when combined with other features, so that *the combination of features* is highly correlated with the target y.
- Example: One-hot-encoding of a categorical feature converts it into a relatively simple 0, 1-valued set of features.
- What if there was a way of converting that set of simple numerical features into a smaller set of derived numerical features *with much the same explanatory power* as the larger set?
- What if that procedure could be applied to any set of numerical features, to derive a smaller set of numerical features, *with much the same explanatory power*?

Forward selection (adding features, one by one, until the error increases) is a simple search strategy.

Feature independence in Multivariate Data

Definition 2 (Covariance)

$\sigma_{12} = E[(X_1 - \mu_1)(X_2 - \mu_2)]$. In words, for two features X_1 and X_2, with means μ_1 and μ_2, respectively, σ_{12} is a measure of the the linear dependence between them. If they are independent, we can show that $\sigma_{12} = 0$.

Definition 3 ((Variance-)Covariance Matrix)

When there are n numeric features, there are $n \times n$ pairs of covariances $\sigma_{ij}, i = 1, \ldots, n; j = 1, \ldots, n$. The resulting covariance matrix is symmetric and diagonally dominant. This matrix captures the covariance structure of the set of n features $\{X_i\}$.

We have seen the correlation matrix, which is a scaled version of the covariance matrix, with elements $\rho_{ij} = \frac{\sigma_{ij}}{\sigma_i \sigma_j}$, all the diagonal elements are 1 and the off diagonal elements satisfy $-1 < \rho_{ij} < 1$. If two features are highly correlated, adding the second into the model does not increase the explanatory power of the model. Therefore, it pays to determine the correlation matrix from the data before building any models.

Multivariate data with correlated measurements

Example 4 (Measles cases, by city, per week from 1948–1985)

This data spans the period before and after the introduction of vaccination for measles (during the mid 1960s). Measles cases are recorded per week in 7 English cities. Although the cities are not adjacent, it is likely that there will be some spatial autocorrelation. Also, by the nature of disease outbreaks, there will also be some temporal autocorrelation per city.

	Date	London	Bristol	Liverpool	Manchester	Newcastle	Birmingham	Sheffield
1	1948-01-17	240	4	51	19	52	84	11
2	1948-01-24	284	3	54	23	34	65	11
3	1948-01-31	340	5	54	31	25	106	4
4	1948-02-07	511	1	89	66	27	142	7
5	1948-02-14	649	3	73	60	47	143	3
6	1948-02-21	766	13	169	87	46	191	6
7	1948-02-28	932	5	212	61	66	208	9
8	1948-03-06	1303	4	283	79	57	290	7
9	1948-03-13	1257	15	285	56	82	310	10
10	1948-03-20	1716	9	279	85	92	425	5
11	1948-03-27	1425	3	424	63	94	481	10

Removing redundant features, based on correlation filters

Pearson Correlation

	London	Bristol	Liverpool	Manchester	Newcastle	Birmingham	Sheffield
London	1.000000	0.474016	0.295005	0.519947	0.520185	0.707410	0.539053
Bristol	0.474016	1.000000	0.228214	0.437572	0.374370	0.546398	0.680336
Liverpool	0.295005	0.228214	1.000000	0.431414	0.482269	0.365078	0.329118
Manchester	0.519947	0.437572	0.431414	1.000000	0.554188	0.472575	0.522391
Newcastle	0.520185	0.374370	0.482269	0.554188	1.000000	0.645766	0.535574
Birmingham	0.707410	0.546398	0.365078	0.472575	0.645766	1.000000	0.690961
Sheffield	0.539053	0.680336	0.329118	0.522391	0.535574	0.690961	1.000000

London-Birmingham has correlation *greater than* 0.7.

Spearman Correlation

	London	Bristol	Liverpool	Manchester	Newcastle	Birmingham	Sheffield
London	1.000000	0.654859	0.399211	0.589346	0.559762	0.764533	0.581148
Bristol	0.654859	1.000000	0.356830	0.598125	0.471088	0.636617	0.613336
Liverpool	0.399211	0.356830	1.000000	0.580160	0.558448	0.383332	0.421292
Manchester	0.589346	0.598125	0.580160	1.000000	0.491076	0.507557	0.577990
Newcastle	0.559762	0.471088	0.558448	0.491076	1.000000	0.591156	0.633679
Birmingham	0.764533	0.636617	0.383332	0.507557	0.591156	1.000000	0.599110
Sheffield	0.581148	0.613336	0.421292	0.577990	0.633679	0.599110	1.000000

London-Birmingham has correlation *greater than* 0.7.

Kendall Correlation

	London	Bristol	Liverpool	Manchester	Newcastle	Birmingham	Sheffield
London	1.000000	0.471882	0.268666	0.417987	0.402433	0.570474	0.416055
Bristol	0.471882	1.000000	0.243417	0.428664	0.331594	0.460080	0.449481
Liverpool	0.268666	0.243417	1.000000	0.411598	0.400088	0.260798	0.291779
Manchester	0.417987	0.428664	0.411598	1.000000	0.346396	0.354931	0.411831
Newcastle	0.402433	0.331594	0.400088	0.346396	1.000000	0.428067	0.463323
Birmingham	0.570474	0.460080	0.260798	0.354931	0.428067	1.000000	0.432066
Sheffield	0.416055	0.449481	0.291779	0.411831	0.463323	0.432066	1.000000

London-Birmingham has correlation *less than* 0.7.

Observations

- Critical level of correlation $\rho^{(\text{crit})} = 0.7$, so one of London or Birmingham can be dropped.
- The Spearman correlations are particularly high, so more correlation might be present.
- The Kendall correlations are inconclusive.

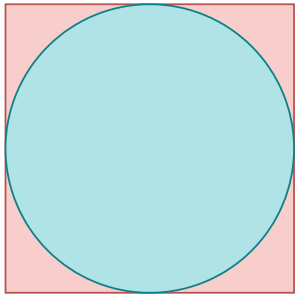
Working with high-dimensional data

Definition 5 (Curse of Dimensionality)

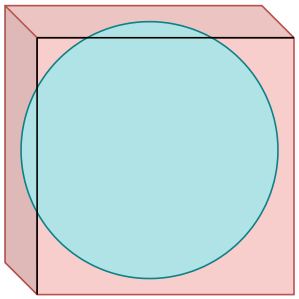
High dimensions do not just require more computing resources and make interpretation more difficult. They also make it more difficult to capture data that samples very high dimensional spaces efficiently. As the dimension d increases, most of the volume of a hypercube is near the corners, not near the centre, where data might be easiest to collect. This makes estimating parameters much more difficult.

- The proof is based on the fact that, as the dimension d tends to infinity, the *ratio* of the volume of the maximum hypersphere inscribed inside the hypercube of the same dimension, tends to 0. Thus there are good reasons to prefer low dimension approximations to high dimensional space.
- In 2D: imagine the largest circle fitting inside a square; ratio is $\frac{\pi r^2}{4r^2} = \frac{\pi}{4} \approx 0.79$.
- In 3D: imagine the largest sphere fitting inside a cube; ratio is $\frac{(4/3)\pi r^3}{8r^3} = \frac{\pi}{6} \approx 0.52$.
- More generally

$$\frac{V_{\text{hypersphere}}}{V_{\text{hypercube}}} = \frac{\pi^{d/2}}{d2^{d-1}\Gamma(d/2)} \to 0 \text{ when } d \to \infty$$



2D: circle in square



3D: sphere in cube

Impact: Harder to collect data that samples high-dimensional space (which is mostly in the "corners"...), so harder to estimate such models. Analogy: "corner cases" when testing software.

Feature reduction

- Sometimes it is possible to use intuition to reduce the dimension, by omitting selected attributes.
- Another possibility is to look for groups of correlated attributes (c.f., *mediation*), such as the London and Birmingham measles cases above, and just choose 1 of these.
- More generally, there are techniques that search for a subspace with specified dimension d' of the attributes that captures most of the variance of the full set of attributes having dimension d, where $d' < d$ (often $d' \ll d$).
- The best known of these techniques is *Principal Components Analysis* (PCA).
- Refer to worked examples of PCA in the lab notebook.

Review of Classification 1

- Classification is one of the most common machine learning tasks
- For regression, the focus is on residuals, for classification it is on the confusion matrix
- Performance metrics are based on ratios and independent of the algorithm used
- Trade-offs are needed: Type 1 vs Type 2 errors and associated classification metrics
- Builds upon all the existing EDA, model building and multivariate analysis we saw before

In classification 2, we introduce 2 new classification algorithms